

Avatar

Part 1: Title, goal statement, and conventions

Project title: Avatar : AI-Driven Synthetic User Testing Platform

Team members: Akshaya Bala Subramani, Suhani Garg, Sarvesh Pol

Goal statement:

Avatar helps UX and accessibility teams identify potential usability barriers early in the design process by running research-based synthetic personas through website tasks within an approved test scope. Its findings support further investigation by human testers and accessibility experts.

Basic input:

The system accepts a target website or prototype URL, a task description, selected synthetic personas, and the permitted pages and actions. Users may also import findings from other testing methods for comparison and prioritization.

Basic output:

The system produces a report showing each persona's task outcome, completed steps, and any barriers encountered. It also provides a combined priority list that merges duplicate issues, identifies affected personas, and ranks validated findings by their impact on task completion.

Diagram conventions:

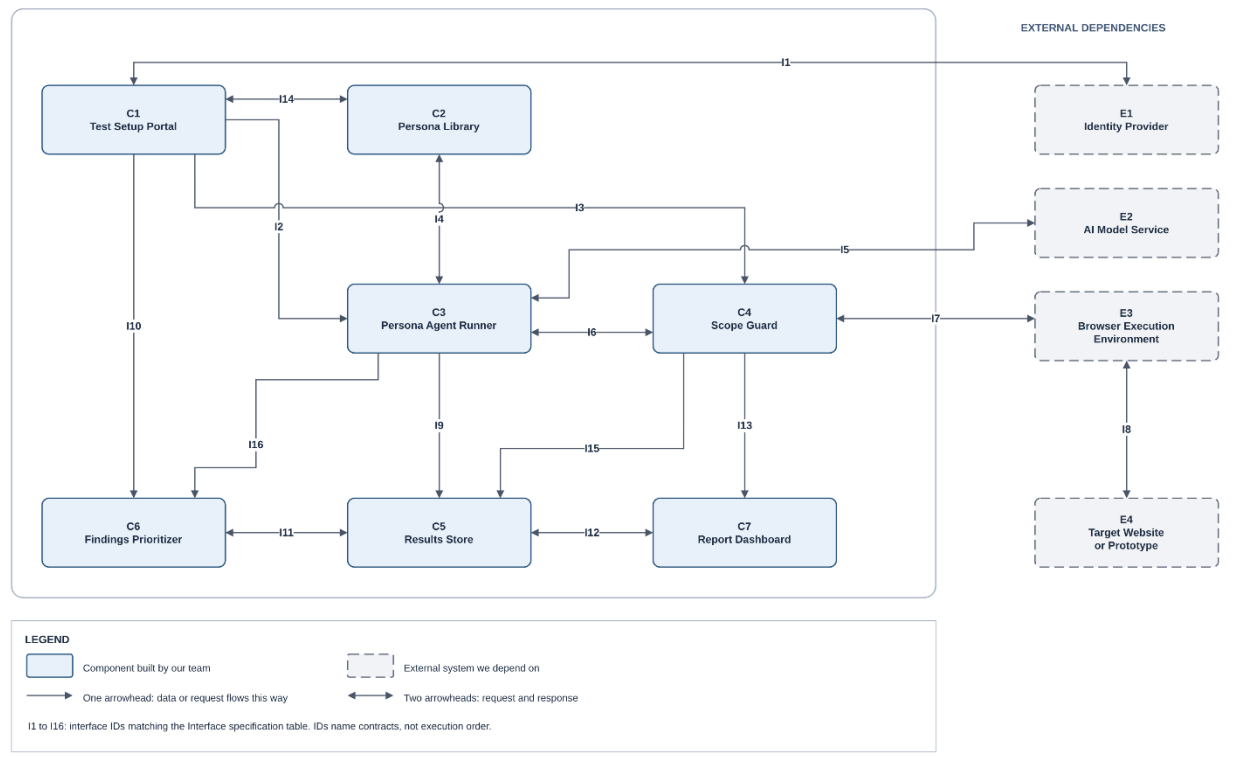
Symbol	Meaning
Solid rounded rectangle	A component developed by our team
Shaded rectangle with a dashed border	An external system that our platform depends on
Arrow	The direction of a request or data transfer
Interface label I1, I2, etc.	A connection defined in the interface specification table; any response is specified in that same row
Data-flow arrow label	The information being transferred and its form at that stage

Both diagrams include the project title, goal statement, and a legend explaining the symbols and arrows used.

Part 2: Block diagram (the D0 diagram)

Avatar: D0 High-Level Design

Goal: Identify potential usability barriers early by running research-based synthetic personas through website tasks within an approved test scope.



Part 3: Component responsibility table

ID	Component	Responsibility	Interfaces in	Interfaces out	User stories	Primary owner
C1	Test Setup Portal	Collects all user-supplied test inputs.	I1, I14	I1, I2, I3, I10, I14	US-01, US-02, US-03, US-05	Akshaya Bala Subramani
C2	Persona Library	Provides persona profiles containing interaction constraints, cited research sources, and review status.	I4, I14	I4, I14	US-04	Suhani Garg
C3	Persona Agent Runner	Coordinates each persona's attempt to complete the configured task.	I2, I4, I5, I6	I4, I5, I6, I9, I16	US-01, US-03, US-04	Sarvesh Pol

ID	Component	Responsibility	Interfaces in	Interfaces out	User stories	Primary owner
C4	Scope Guard	Enforces the permitted pages and actions during test execution.	I3, I6, I7	I6, I7, I13, I15	US-05	Sarvesh Pol
C5	Results Store	Preserves test evidence, findings, rankings, and scope audit records.	I9, I11, I12, I15	I11, I12	US-01, US-02, US-03, US-05	Suhani Garg
C6	Findings Prioritizer	Produces a deduplicated priority list from validated findings across testing methods.	I10, I11, I16	I11	US-02, US-04	Suhani Garg
C7	Report Dashboard	Presents test outcomes, prioritized findings, and scope alerts to users.	I12, I13	I12	US-01, US-02, US-03, US-05	Akshaya Bala Subramani

External systems

ID	External dependency	Role
E1	Identity Provider	Authenticates platform users.
E2	AI Model Service	Proposes the persona's next action from the supplied task and page state.
E3	Browser Execution Environment	Executes permitted browser actions and returns page observations.
E4	Target Website or Prototype	Provides the pages and interactions being evaluated.

Part 4: Interface specification table

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
I1	C1 ↔ E1	client_id: string; redirect_uri: string; state: string	id_token: string; user_id: string; role: string	Authentication parameters and JSON token response	OpenID Connect over HTTPS	E1 rejects failed authentication. C1 rejects invalid or expired tokens, displays a sign-in error, and prevents test

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
						submission.
I2	C1 → C3	job_id: UUID; product_id: UUID; target_url: string; task: string; persona_ids: string[]; scope_id: UUID	job_id: UUID; status: string; error_message: string or null	JSON	HTTPS REST — POST /jobs	C3 rejects invalid settings. A broken or inaccessible target stops the test within 30 seconds; C1 displays the URL and reason while retaining the settings.
I3	C1 → C4	scope_id: UUID; allowed_urls: string[]; blocked_actions: string[]	scope_id: UUID; saved: boolean; errors: string[]	JSON	HTTPS REST — PUT /scopes/{scope_id}	C4 rejects an empty allowed list or invalid rules, identifies the offending field, and leaves the previous scope unchanged.
I4	C3 ↔ C2	persona_id: string	persona_id: string; interaction_constraints: {allowed_input_modes: string[], requires_accessible_labels: boolean, minimum_text_contrast_ratio: number or null}; assistive_technology: string; sources: string[]; reviewed: boolean	Typed, JSON-serializable persona profile object.	In-process function call within the backend.	C2 reports a missing profile. C3 stops that persona's run if it is unavailable; missing sources or review status cause all its findings to be marked unvalidated and excluded by C6.
I5	C3 ↔ E2	persona_profile: object; task: string; page_state: {url: string, accessibility_tree: string,	action_type: string; target: string or null; test_value: string or null; reason: string	JSON	HTTPS API request/response	C3 validates the returned action. Timeout, invalid output, or rate limiting triggers at most two

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
		focused_element: string or null}; step_history: object[]				retries; continued failure ends that persona's run with earlier evidence preserved. Two retries covers a brief timeout or rate limit without eating into the 10-minute task budget.
I6	C3 ↔ C4	job_id: UUID; scope_id: UUID; proposed_action: object	allowed: boolean; reason: string; page_state: {url: string, accessibility_tree: string, focused_element: string or null} or null; execution_status: string	Typed, JSON-serializable action-check request and execution-result objects.	In-process function call within the backend.	C4 checks scope before execution through I7. A denied action pauses the run and produces an I13 alert; guard failure blocks execution.
I7	C4 ↔ E3	job_id: UUID; approved_action: object; scope_policy: object	current_url: string; page_state: {url: string, accessibility_tree: string, focused_element: string or null} or null; execution_status: string; error_message: string or null	JSON command and observation records	Local inter-process communication (IPC) using JSON request/response messages over standard input/output.	E3 applies the supplied restrictions to navigation, redirects, and prohibited requests. C4 handles execution errors, pauses unsafe actions, and reports page-load failure within the 30-second target-access budget.
I8	E3 ↔ E4	method: string; url: string; headers: object;	status_code: integer; headers: object; body: string	HTTP messages containing	HTTPS	E3 reports network failures, HTTP errors, or

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
		body: string or null	or binary	HTML, styles, scripts, images, or JSON		login requirements through I7. C4 returns the failure to C3 through I6; C3 stops an inaccessible-target run.
I9	C3 → C5	job_id: UUID; persona_id: string; step_no: integer; action: object; outcome: enum {pass, fail, blocked, error}; finding: {issue_id: UUID, page_url: string, element: string, description: string, severity: enum {blocks_task, slows_task, minor}, wcag_criterion: string or null} or null; validated: boolean	record_id: UUID; saved: boolean	Typed, JSON-serializable step-result object and save acknowledgement.	In-process function call to the Results Store within the backend.	C5 reports failed writes. C3 retries at most three times using the same record identity, then marks the run incomplete without deleting saved steps. Three retries covers a short storage hiccup, and reusing the same record ID means a retry can't create duplicates.
II 0	C1 → C6	product_id: UUID; file: binary containing issue_id: string, source_method: string, page_url: string, element: string, severity: string	accepted_count: integer; errors: string[]	CSV or JSON file; JSON response	HTTPS REST — multipart POST /findings/import	C6 validates the whole file before saving. Unsupported format, missing fields, or invalid values reject the entire import with zero additions; C1 displays the reason.
II 1	C6 ↔ C5	operation: string; product_id:	stored_findings: object[]; saved:	Typed, JSON-serializable	In-process function calls	C6 excludes unvalidated

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
		UUID; findings: object[] or null; ranked_issues: object[] or null	boolean; ranking_version: UUID or null	finding lists, ranked-issue lists, and storage responses.	to read and write findings and rankings through the Results Store within the backend.	findings, merges duplicates, and ranks eligible issues. If reading or saving fails, C5 retains the previous complete ranking and C6 reports the failure.
I1 2	C7 ↔ C5	job_id: UUID or null; product_id: UUID or null	report: {job_id: UUID, persona_results: {persona_id: string, outcome: enum {pass, fail, blocked, error}, completed_steps: integer, blocked_step: integer or null, validated: boolean}[], findings: object[]; ranked_issues: object[]; scope_events: object[]; status: string	JSON	HTTPS REST: GET /reports/{job_id} or GET /products/{product_id}/issues	C5 returns a not-found or unavailable response. C7 displays the error without presenting missing data as a successful report.
I1 3	C4 → C7	event_id: UUID; job_id: UUID; blocked_url: string or null; action_type: string; reason: string; timestamp: string	event_id: UUID; received: boolean	JSON	Secure WebSocket event and acknowledgment	C4 sends the scope alert within five seconds of the blocked attempt. If delivery fails, the event remains stored through I15 and C7 retrieves it through I12 when connected again.

ID	From → To	Inputs: named and typed	Outputs: named and typed	Data format	Protocol	Error behavior and handler
I1 4	C1 ↔ C2	persona_id: string or null	profiles: object[] containing persona_id: string, name: string, assistive_technology: string, sources: string[], reviewed: boolean	JSON	HTTPS REST — GET /personas or GET /personas/{persona_id}	C2 reports unavailable profiles. C1 prevents selection of missing profiles and warns when sources or review are missing; such selections are visibly marked unvalidated.
I1 5	C4 → C5	event_id: UUID; job_id: UUID; url: string; action_type: string; allowed: boolean; reason: string; timestamp: string	event_id: UUID; saved: boolean	Typed, JSON-serializable audit-record object and save acknowledgment.	In-process function call to save an audit record through the Results Store within the backend.	C4 requires a durable audit record before allowing a new action. If C5 cannot save it, C4 pauses execution and alerts C7 through I13.
I1 6	C3 → C6	job_id: UUID; product_id: UUID; run_status: string	accepted: boolean; ranking_status: string; error_message: string or null	Typed, JSON-serializable ranking-request object and status response.	In-process function call within the backend to request a ranking update.	C3 requests ranking after saving the available test evidence. If C6 cannot update the ranking, the test evidence remains available and the previous ranking is retained.

Example payload I2: submitting a test job

```
{
  "job_id": "7c1e2b90-4d3a-4f6e-9a21-0b5c8e7d1f42",
  "product_id": "be52ab7d-3a64-4dc6-a3f2-695d75dc92da",
  "target_url": "https://staging.example.com/checkout",
  "task": "Add one item to the cart and reach the payment page without submitting payment",
  "persona_ids": [
```

```

"screen-reader-01",
"low-vision-02",
"keyboard-only-03"
],
"scope_id": "3f9a6c12-8b4e-4d71-a0c5-2e6f9b8d4a17"
}

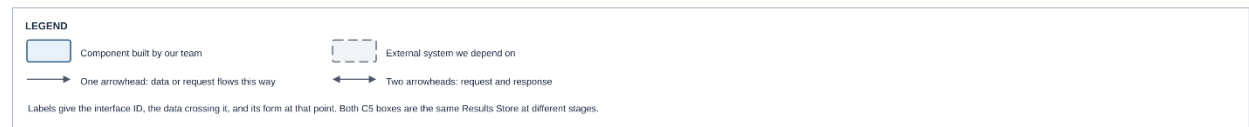
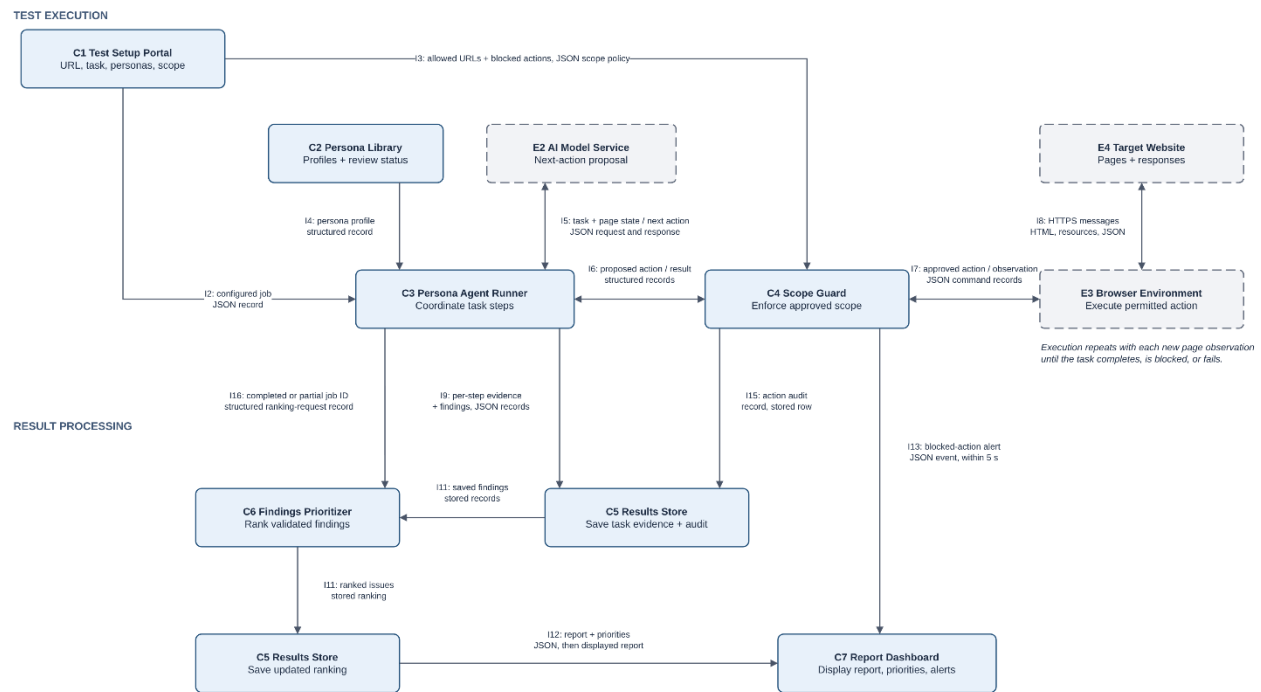
```

Part 5: Data-flow diagram

Flow 1 Persona test run

Avatar: Flow 1, Persona Test Run

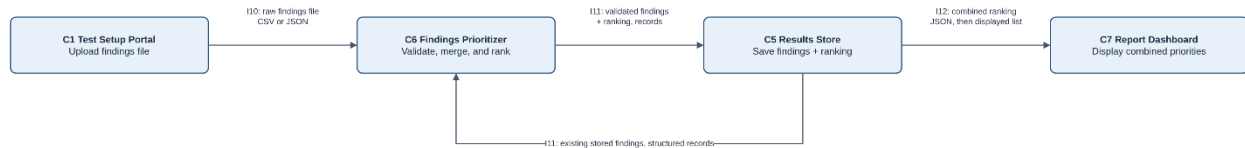
Goal: Identify potential usability barriers early by running research-based synthetic personas through website tasks within an approved test scope.



Flow 2 Findings import and ranking

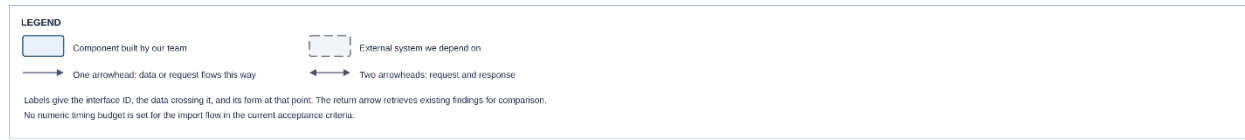
Avatar: Flow 2, Findings Import and Ranking

Goal: Identify potential usability barriers early by running research-based synthetic personas through website tasks within an approved test scope.



IMPORT FAILURE: reject the whole file if its format or required fields are invalid. Add zero issues and keep existing findings unchanged.

RANKING RULE: findings from unsourced or unreviewed personas are marked unvalidated and excluded from ranking.



Timing requirement	Source
A broken, expired, or inaccessible login-protected URL causes the test to stop within 30 seconds.	AC-01.2
A blocked-action alert reaches the administrator's dashboard within 5 seconds.	AC-05.2
A task-completion report for at least one screen-reader persona is available within 10 minutes.	AC-03.1
A barrier report for at least three personas is available within 15 minutes.	AC-01.1

Part 6: Architecture pattern and justification

Avatar uses a combination of client-server, layered, and pipeline architecture patterns.

The client-server pattern governs access to the platform. Users configure tests through the Test Setup Portal (C1) and view results through the Report Dashboard (C7). The backend manages persona profiles, test execution, scope enforcement, findings prioritization, and stored results.

The layered pattern separates presentation, application logic, and data access. C1 and C7 form the presentation layer; C2, C3, C4, and C6 provide application logic; C5 provides access to stored evidence and results. These are logical boundaries and do not require a separate deployed service for every component.

The pipeline pattern governs the processing stages. During a test, the system loads a persona profile, obtains a proposed action, checks its scope, executes the permitted action, and records the outcome. Page

observations feed back into the next action cycle. Imported findings follow a second pipeline: validate, combine, prioritize, store, and display.

Justification against the five criteria

Criterion	Justification
Fit to the problem	Client-server supports a shared testing platform for researchers, QA engineers, and administrators. Layered boundaries separate user interaction from execution and storage. The pipelines make scope checks and findings validation explicit stages, supporting US-01 through US-05.
Team skills	Akshaya has a product background, so she owns the user-facing parts: the Test Setup Portal (C1) and Report Dashboard (C7). Suhani has worked on AI applications, which fits the Persona Library (C2), Findings Prioritizer (C6) and Results Store (C5). Sarvesh handles the backend side, so he owns the Persona Agent Runner (C3) and Scope Guard (C4). A layered client-server design lets each of us work in our own layer without waiting on the others. Microservices would add infrastructure work none of us has much experience with.
Performance and timing	Internal calls between the runner and Scope Guard avoid an extra network dependency for each scope check. External model and website calls remain likely sources of delay, so execution needs bounded waits and retries. The required budgets are 30 seconds to stop an inaccessible-target test, 10 minutes for a task result, 15 minutes for a report covering at least three personas, and 5 seconds for a scope alert. We'll check these budgets against a real staging site once the runner works end to end.
Scalability	We'll start by allowing only a few test runs at once. Each browser session uses a lot of memory, and every model call costs money. Additional runner capacity can be introduced later while preserving the same component interfaces. Shared results storage allows findings from different runs and testing methods to be combined.
Hardware constraints	The platform requires ordinary computing resources capable of running a backend and browser sessions; it does not require custom sensors or actuators. Browser sessions consume memory and processing capacity, so concurrency must match the available host. Our Week 3 constraints don't set a hosting or API budget yet, so we'll agree on a monthly cap as a team before the first full test run.

Week 3 constraints

The architecture supports the security constraints by placing scope enforcement before browser execution and supplying the browser environment with the permitted pages and actions. Tests must use synthetic accounts and non-sensitive data rather than real credentials or payment information. Page observations sent to the external model service must also be limited to the information needed for testing.

The Persona Library records research sources and review status to support responsible representation of accessibility needs. Findings from unreviewed or unsourced personas remain labeled unvalidated and are excluded from priority ranking. Reports present potential barriers for further investigation rather than claiming to replace testing with real users or accessibility experts.

The presentation layer must support keyboard operation, accessible labels, and readable reports. Separating persona profiles from execution logic also allows the platform to support different interaction constraints and user contexts without redesigning the overall structure.

Rejected alternative

We rejected microservices for the initial version. Deploying each component independently would add network communication, deployment configuration, and monitoring work. For a three-person team, that's more setup than it's worth. One backend with clear internal boundaries gives us the same separation, and we can split out the runner later if it needs to scale.

Part 7: Decision log

ID	Decision	Alternatives considered	Reason for choosing it	Requirement link
DL-1	Require every browser action to pass through the Scope Guard before execution, with scope restrictions also applied in the browser environment.	Let the runner enforce scope alone; rely only on the target site's restrictions.	Scope checks shouldn't depend on the same code that decides the next action. Browser-level limits also catch redirects and background requests the runner never sees.	US-05; AC-05.1, AC-05.2
DL-2	Maintain a shared Persona Library containing interaction constraints, research sources, and review status.	Embed personas directly in agent instructions; accept unrestricted user-created personas.	Teams need to see where a persona's behavior comes from. A shared library makes that visible and lets us handle unreviewed or unsourced personas the same way every time.	US-04; AC-04.1, AC-04.2
DL-3	Preserve step evidence as the test progresses.	Save results only after the entire task finishes.	If a run fails on step 6, we still want steps 1 to 5. Saving as we go also lets the report show exactly where the persona got stuck.	US-03; AC-03.2
DL-4	Validate an entire imported findings file before adding any of its issues.	Accept valid rows while rejecting invalid rows; add findings before checking them.	A partial import leaves a list nobody can trust. All or nothing keeps the existing findings safe and matches AC-02.2.	US-02; AC-02.2
DL-5	Keep unvalidated persona findings visible in reports but exclude them from priority ranking.	Rank all findings; discard unvalidated findings entirely.	Hiding them loses information. Ranking them gives weight to evidence we can't back up.	US-04; AC-04.2

ID	Decision	Alternatives considered	Reason for choosing it	Requirement link
DL-6	Use one backend with logical layers and processing pipelines for the initial version.	Deploy each component as a microservice; build a standalone script without a shared interface.	Microservices would be too much infrastructure for three people, and a standalone script would give us no shared interface. One backend still lets each of us own clear parts.	Architecture justification; US-01–US-05
DL-7	Combine duplicate findings into one issue and prioritize validated issues by their impact on task completion.	Keep separate lists for each testing method; rank only by the number of occurrences.	Teams care about which issue to fix first, not which tool found it. Ranking by impact puts task-blocking issues at the top, which a simple count wouldn't.	US-02; AC-02.1